

Canva Grow 营销内容合规层 PRD 2

1.1 文档信息

项目	内容
产品名称	Canva Grow Marketing Compliance Layer
中文名称	Canva Grow 营销内容合规层
当前阶段	MVP / Demo 验证
首期市场	Australia
首期行业	Beauty / Cosmetics / AUST L Listed Medicines
首期内容类型	文本营销内容
核心能力	发布前营销合规检测、缺失义务识别、风险解释、最小改写
产品形态	嵌入 Canva Grow 内容生成链路的 Compliance Layer
核心原则	Ask, don't guess / Detect what is missing, not only what is wrong / Minimise false positives

1. 项目背景

1.1 业务背景

Canva Grow 已能够通过 AI 帮助用户快速生成广告概念及营销内容。

从当前 Grow 实测流程来看，用户通常经历：

代码块

- 1 输入产品或营销需求
- 2 → Grow 生成多个广告概念
- 3 → 用户选择广告方向
- 4 → Grow 继续生成广告内容
- 5 → 用户修改 / 使用内容

现有 AI 能力解决的是：

“如何更快生成营销内容。”

但生成内容之后，用户仍然需要自行判断：

“这条广告能不能安全地发出去。”

这一问题在 Beauty、Cosmetics、Therapeutic Goods 等存在明确广告约束的领域更加明显。

原调研同样发现，中小企业通常缺少专职法务，而现有方案又存在敏感词工具不理解语境、通用 AI 容易误杀、平台工具缺乏澳洲本地法规适配等问题。

因此，本项目希望在 Canva Grow 的**内容生成与用户继续使用之间**增加一层营销合规检查。

核心价值不是让用户主动想起：

“我要不要去检查一下法规？”

而是在：

用户没有想到要问的那一刻，系统自己出现。

1.2 问题定义

当前营销内容合规问题主要存在三种类型。

类型一：广告说了不应该说的话

例如：

代码块

```
1  No side effects.  
2  100% natural.  
3  Guaranteed results.
```

这一类风险相对容易被敏感词或通用 LLM 发现。

类型二：广告说的话是否成立，取决于系统看不到事实

例如：

代码块

```
1  Clinically proven to improve skin.
```

系统无法仅根据广告文本判断：

- 用户是否真正持有临床证据；
- 证据是否能够支持该具体 claim；

- 产品是否属于当前规则范围。

如果模型直接判断“违规”或“合规”，都会产生较高误判风险。

因此正确行为不是猜测，而是：

代码块

```
1 Needs your input
2 → 向用户索取缺失信息
3 → 再继续判断
```

类型三：广告没有说错，但漏掉了依法必须提供的信息

这是本项目最重要的切入点。

传统敏感词工具通常只寻找：

“你说了什么危险内容？”

但真实的广告合规问题也可能来自：

“法规要求你必须说什么，但 AI 没有生成。”

例如治疗商品广告存在 mandatory information / mandatory statements 等明确义务。

这类问题具有以下特点：

- 广告每句话本身可能都没有明显问题；
- 不包含敏感词；
- 通用生成模型容易遗漏；
- 平台安全层也不一定能够发现；
- 规则相对结构化；
- 可以通过法规知识进行明确验证。

因此，该项目不仅检测：

代码块

```
1 What the AI said
```

也检测：

代码块

2. 为什么做

2.1 现有方案问题

当前替代方案主要包括：

方案	优势	主要问题
ChatGPT 类通用 AI	理解上下文、可以解释和改写	澳洲法规适配不稳定；可能误杀；结果一致性不足
国内敏感词工具	快、成本低	以关键词为中心；澳洲法规基本不适配
平台 Policy Checker	对渠道政策较强	主要解决平台审核，不等同于当地法律合规
人工审核	准确性相对较高	成本高、速度慢、难覆盖大量 AI 内容
法规网站	官方可靠	用户需要主动搜索，并自行理解和映射至具体广告

原调研验证的主要问题也是：词库不理解语境、AI 容易误杀、平台工具不熟悉澳洲当地法规。

2.2 为什么优先选择 Beauty

早期调研同时考察了金融与健康美容。

当前 MVP 最终选择 Beauty / Cosmetics / AUST L Listed Medicines，而不以金融作为一期切口。

主要原因包括：

1. Beauty 与 Canva Grow 的营销内容生成场景结合更自然；
2. 美妆用户存在大量 AI 广告生成需求；
3. 一部分 therapeutic goods 规则具有明确、可验证、可引用的结构化义务；
4. 可以同时测试“错误表达”“缺失信息”“需要证据”三类 AI 判断；
5. 更适合构造大量 Hard Negative Case；
6. 当前项目团队对该领域场景理解程度高于金融领域。

这不是判断金融需求较弱，而是 MVP 的 Scope Decision。

2.3 产品机会

本项目不重新构建 Canva Grow 的生成能力，而是在现有工作流中增加：

代码块

1

Generate

2

→ Compliance Check

3

→ Fix / Confirm

4

→ Continue

这与调研中提出的“在 Canva 生成和发布之间增加发布前预检”方向一致。

fileciteturn0file0L82-L90

3. 产品需求定义

3.1 产品定位

Canva Grow Marketing Compliance Layer 是一个嵌入 Canva Grow 内容生产流程的 **AI 营销合规辅助层**。

系统根据：

代码块

1

投放法域

2

+

3

产品监管类型

4

+

5

广告内容

6

+

7

适用法规

8

+

9

案例

10

+

11

误杀样本

自动检测用户即将使用的营销内容，并输出明确的下一步操作。

产品不以“风险有多高”为主要输出，而是回答：

用户现在应该做什么？

统一使用三个 Action Status：

Ready to run

代码块前选择的合规范围内，
2 未发现需要用户立即修改的明确问题。

不等同于：

代码块

1 Legally approved
2 100% compliant

Fix before running

表示：

当前已有信息足够，并发现明确的法规义务未满足或存在应修改内容。

用户在继续之前应处理该问题。

Needs your input

表示：

当前判断客观依赖系统无法获得的信息。

例如：

- 产品监管类型不确定；
- 缺少 supporting evidence；
- 产品身份与 claim 产生冲突；
- 缺少 AUST L / 产品信息；
- 某一判断依赖用户掌握的业务事实。

这不是“AI 信心不足”的模糊状态，而是正式的产品状态。

3.2 目标用户

Primary User

澳洲中小型 Beauty / Skincare / Wellness 品牌的：

- Founder

- Marketing Manager
- Social Media Manager
- Content Creator
- E-commerce Operator

典型特征：

- 使用 Canva 制作营销内容；
- 使用 AI 生成广告；
- 没有专职法律或 Compliance 团队；
- 知道广告存在监管问题，但无法持续跟踪法规；
- 更想知道“怎么修改”，而不是学习法规原文。

Secondary User

后续版本包括：

- Marketing Agency
- Freelancer
- 多品牌营销团队
- 跨境营销团队
- 企业 Compliance Reviewer

调研中的用户需求也主要集中在“准确发现风险、解释原因并给出仍保留营销效果的改写”。

3.3 用户痛点

Pain Point 1：用户不知道自己需要检查

当前大部分 Compliance 工具需要用户主动：

代码块

- 1 复制广告
- 2 → 打开工具
- 3 → 粘贴
- 4 → 主动询问

但真正危险的是：

用户根本没有意识到这条内容需要检查。

因此 Compliance Layer 必须嵌入 Grow 工作流，而不是依赖用户主动发起。

Pain Point 2：关键词检测无法判断上下文

例如：

代码块

```
1 This is our team's favourite formula.
```

该表达不应仅因为：

代码块

```
1 formula
```

就被识别为治疗功效相关内容。

误杀过多会产生：

代码块

```
1 False Positive
2 → Warning Fatigue
3 → 用户不再相信系统
```

Pain Point 3：通用 AI 倾向“宁可多拦”

现有 LLM 在法律风险场景中容易采取过度谨慎策略。

这会导致：

代码块

```
1 可能有问题
2 ≈
3 直接提示违规
```

从而把真实的：

代码块 信息不足

错误变成：

代码块

1 High Risk

本产品需要明确区分：

代码块

1 需要修改

2 ≠

3 需要补信息

Pain Point 4：用户不知道产品应该使用哪个规则集

Beauty 产品监管边界本身可能复杂。

因此系统不能在后台静默判断：

代码块

1 这一定是 Cosmetic

或：

代码块

1 这一定是 Therapeutic Good

错误品类：

代码块

1 → 错误规则集

2 → 错误判断

3 → False Positive / False Negative

因此分类必须：

可见、可确认、可修改。

Pain Point 5：法规问题不仅来自“错误内容”

用户现有工具更多寻找：

代码块

1

禁止词

2

风险 claim

但对于部分广告，真正的问题可能是：

代码块

1

Required Information Missing

用户很难主动意识到这一类错误。

Pain Point 6：用户知道有问题，但不知道改了什么

直接重新生成整条广告会破坏：

- Brand Tone；
- Campaign Intent；
- 原有表达；
- 用户对 AI 的控制感。

因此 MVP 不追求多版本生成，而追求：

Minimal Rewrite + Delta

明确展示：

代码块

1

修改了哪一处

2

为什么改

3

增加了什么

4

删除了什么

3.4 核心使用场景

场景一：Grow 生成广告后的自动预检

用户完成广告生成。

系统自动进入：

代码块

1 Compliance Check

而不是等待用户主动点击另一个产品。

目标

做到：

在用户准备继续的时候，而不是在用户想到法规的时候，进行检查。

场景二：确认产品监管类型

系统自动带入：

代码块

1 Country: Australia

2 Product category: Beauty

如果系统能够根据网站、Canva Brand 信息或已有数据推断产品类型，可以预填。

但必须在 UI 明确展示：

代码块

1 Product regulation

2 Cosmetic ▼

用户一键可改。

首次或低置信度情况下询问：

代码块

- 1 How is this product regulated in Australia?
- 2
- 3 ○ Cosmetic
- 4 ○ AUST L listed medicine
- 5 ○ Other therapeutic good
- 6 ○ I'm not sure

原则：

| Auto-fill ≠ hidden assumption.

场景三：系统检测缺失 mandatory information

用户广告本身不存在明显违规词。

系统根据产品 Context 判断：

代码块

- 1 Product = applicable therapeutic good
- 2 +
- 3 Advertising requirements
- 4 +
- 5 Generated copy

发现 required information 未出现。

系统返回：

代码块

- 1 Fix before running

并明确告诉用户：

代码块

- 1 What's missing
 - 2 Why it is required
 - 3 What needs to be added
 - 4 Source
-

场景四：系统发现需要证据的 Claim

例如：

代码块

```
1 Clinically proven.
```

如果系统没有证据：

不能直接输出：

代码块

```
1 Fix before running
```

而应：

代码块

```
1 Needs your input
```

并询问：

| Do you have evidence supporting this claim?

用户：

代码块

```
1 Yes
2 No
3 Not sure
```

再进入后续判断。

场景五：产品身份与文案发生冲突

例如：

代码块

```
1 Product type = Cosmetic
```

但广告中出现明显 therapeutic-style claim。

系统不直接：

代码块

1 重新分类产品

也不直接：

代码块

1 判定违反 TGA

而输出：

代码块

1 Needs your input

提示：

This claim may affect which regulatory framework applies.
Please confirm how the product is regulated before continuing.

该功能定义为：

Regulatory Boundary Trigger

场景六：用户不同意系统判断

系统提示：

代码块

1 Fix before running

用户认为系统判断错误。

允许用户继续，但必须进行二次确认。

系统记录：

代码块

- 1 Who
- 2 When
- 3 Which flag
- 4 Original copy
- 5 Reason
- 6 Final copy
- 7 Rule version

3.5 核心功能

3.5.1 Automatic Compliance Trigger

用户场景

Grow 已生成广告。

系统行为

自动触发 Compliance Check。

设计原则

用户无需：

代码块

- 1 主动打开 Checker

Compliance 是内容流程中的默认步骤。

3.5.2 Context Confirmation

Input

代码块

- 1 country: Australia
- 2 industry: Beauty
- 3 product_regulation:
- 4 - cosmetic
- 5 - aust_l_listed_medicine
- 6 - other_therapeutic_good

```
7 - unknown
8 channel: optional
```

规则

自动识别结果必须：

- visible;
- editable;
- user-confirmable。

禁止：

代码块

```
1 后台静默分类并直接调用规则
```

3.5.3 Claim Extraction

AI 将文本拆分为 Claim。

例如：

代码块

```
1 Visible results in 14 days.
```

输出：

代码块

```
1 claim_text: "Visible results in 14 days"
2 claim_type: time_bound_effect_claim
3 time_condition: "14 days"
```

Claim Extraction 不负责最终法规判断。

3.5.4 Mandatory Obligation Check

这是 MVP 最核心能力之一。

负责回答：

“依法必须出现的内容有没有出现？”

逻辑：

代码块

```
1 Product Context
2 ↓
3 Applicable Regulation
4 ↓
5 Required Information
6 ↓
7 Generated Copy
8 ↓
9 Present / Missing / Not Applicable
```

如果：

代码块

```
1 Requirement Applicable = true
2 Requirement Present = false
```

输出：

代码块

```
1 Fix before running
```

3.5.5 Semantic Claim Check

负责判断：

已经出现的表达是否存在 claim 风险。

例如：

代码块

```
1 No side effects.
2 Guaranteed results.
3 Clinically proven.
```

AI 需要结合上下文，而不是单词匹配。

3.5.6 Regulatory Boundary Trigger

当：

```
代码块
1 Current product regulation
```

与：

```
代码块
1 Advertising claim
```

可能发生监管框架冲突时：

```
代码块
1 Needs your input
```

不允许模型静默切换规则集。

3.5.7 Evidence-dependent Check

如果 Claim 是否成立取决于外部证据：

```
代码块
1 Needs your input
```

支持状态：

```
代码块
1 evidence_status:
2   - available
3   - unavailable
4   - user_not_sure
```

MVP 不自动判断复杂临床研究质量。

3.5.8 Compliance Result

结果结构统一为：

代码块

```
1  status:
2    - ready_to_run
3    - fix_before_running
4    - needs_your_input
5
6  flagged_text:
7  issue_type:
8  reason:
9  applicable_rule:
10 official_source:
11 required_action:
12 suggested_rewrite:
13 delta:
```

3.5.9 Minimal Rewrite

MVP 每个问题只给：

代码块

```
1  1 条 Recommended Rewrite
```

避免多个版本稀释 Rewrite Adoption Rate。

必须同时输出：

代码块

```
1  Delta
```

例如：

代码块

- 1 Changed:
- 2 "No side effects"
- 3 →
- 4 removed absolute safety claim

或：

代码块

- 1 Added:
- 2 required disclosure

3.5.10 Re-check

用户修改后可以：

代码块

- 1 Re-check

系统重新运行完整 Decision Logic。

不得直接假设：

代码块

- 1 使用 AI 建议 = 已解决问题

3.5.11 Override & Audit Trail

对于：

代码块

- 1 Fix before running

用户仍可继续。

但必须二次确认。

记录字段：

代码块

```
1 reviewer_id:
2 timestamp:
3 flag_id:
4 original_claim:
5 system_status:
6 system_reason:
7 rule_id:
8 rule_version:
9 user_action:
10 override_reason:
11 final_copy:
```

Override Reason

建议 MVP 使用：

代码块

```
1 I have supporting evidence
2 Reviewed internally
3 I disagree with this flag
4 Other
```

是否将“理由必填”作为正式上线要求，当前暂定为：

Required for Fix before running override

3.5.12 Dispute

需要严格区分：

代码块

```
1 用户保留原文
```

和：

代码块

1 用户认为系统错了

埋点分别为：

代码块

```
1  override_confirmed
2  flag_disputed
```

`flag_disputed` 才进入 False Positive Review Queue。

3.5.13 异常与兜底

Case 1：产品分类未知

代码块

```
1  Needs your input
```

Case 2：无法匹配规则

不允许 AI 自行补规则。

输出：

代码块

```
1  We don't have enough regulatory coverage
2  to assess this claim yet.
```

Case 3：法规来源缺失

如果没有可靠 Source：

不得输出：

代码块

1 Fix before running

可降级：

代码块

1 Needs your input / Unable to verify

Case 4：系统执行失败

统一提示：

代码块

1 Compliance check is temporarily unavailable.
2 Your content has not been assessed.

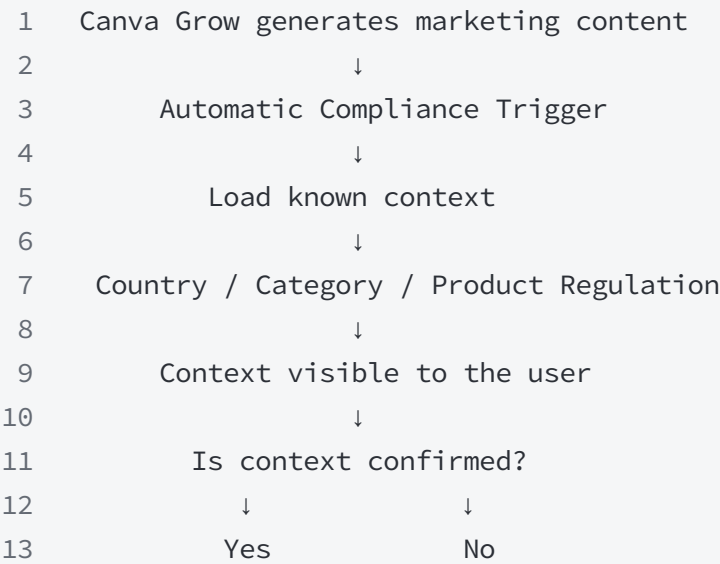
禁止失败后显示：

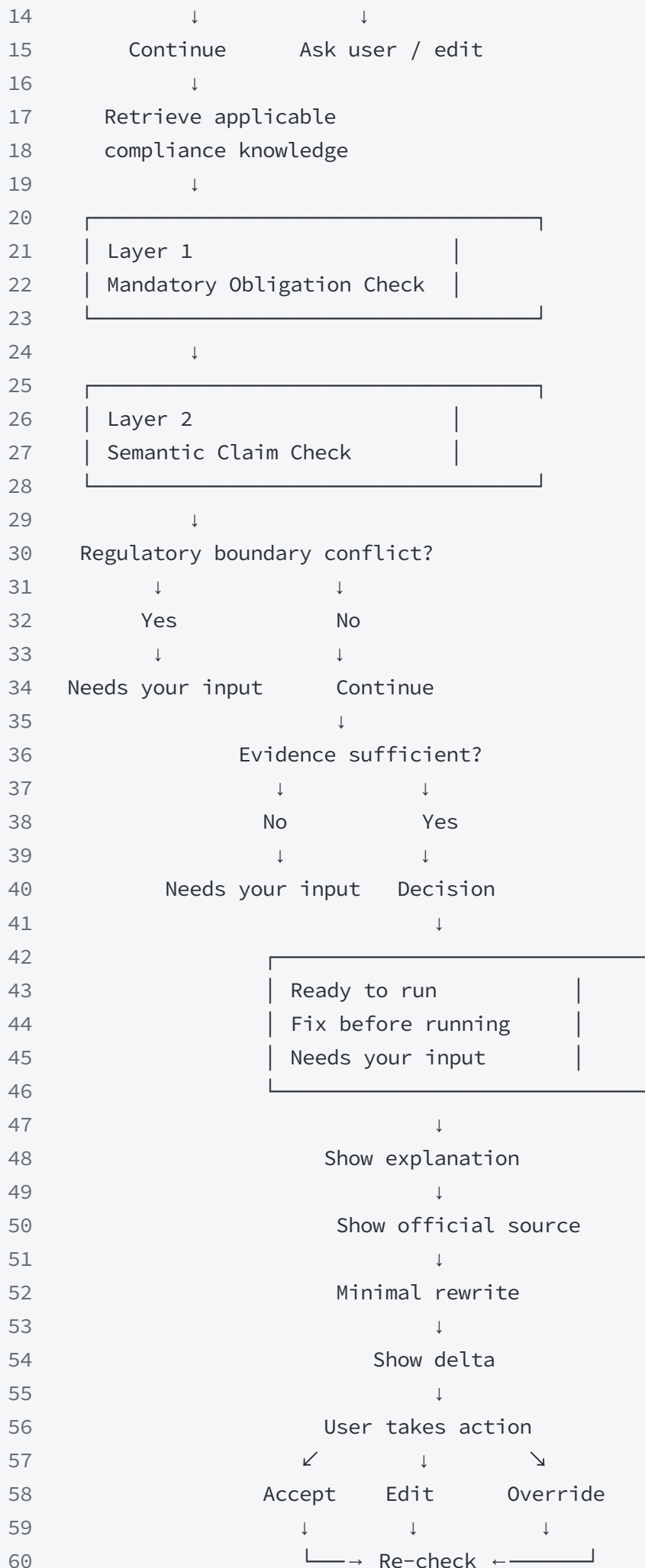
代码块

1 Ready to run

3.6 用户流程

代码块





61

↓

62

Save audit data

63

↓

64

Continue Grow workflow

参考 AI 助手 PRD 的设计方式，本项目同样需要把“识别—判断—参数缺失—执行—失败兜底”作为完整流程，而不是只描述 AI 输出页面。

filecite

turn1file0

L108-L118

filecite

turn1file0

L193-L198

3.7 MVP 范围

P0 — 本期

Geography

代码块

1

Australia

Industry

代码块

1

Beauty

Product Scope

代码块

1

Cosmetics

2

AUST L Listed Medicines

其他 therapeutic goods 仅保留分类接口，不保证完整检测覆盖。

Regulation Scope

首期聚焦：

代码块

1 Australian jurisdiction compliance

暂不以 Meta 等渠道政策作为核心检测对象。

设计约束：

└ Jurisdiction 与 Channel 必须作为两个独立维度。

不得合并为：

代码块

1 Overall compliance score = 80

Content Scope

代码块

1 Text only

不包含：

- 图片内容；
- Before/After 图片真实性；
- 视频；
- 音频；
- 包装视觉；
- 落地页整体审核。

P0 功能

代码块

- 1 Automatic compliance trigger
- 2 Context confirmation
- 3 Claim extraction
- 4 Mandatory obligation check
- 5 Semantic claim check
- 6 Regulatory boundary trigger

- 7 Needs-your-input routing
- 8 Official source citation
- 9 Minimal rewrite
- 10 Delta
- 11 Re-check
- 12 Override
- 13 Audit log
- 14 Dispute feedback
- 15 Basic fallback

P1

代码块

- 1 ARTG / product information deeper integration
- 2 Evidence upload
- 3 Multiple claims batch review
- 4 Multiple copy variants
- 5 Brand tone preservation
- 6 Team review
- 7 Compliance history
- 8 Rule-change alerts

P2

代码块

- 1 Image compliance
- 2 Multimodal advertising review
- 3 Meta / Google / TikTok channel policies
- 4 More Australian industries
- 5 More countries
- 6 Enterprise reporting
- 7 Cross-market compliance comparison

3.8 AI 能力说明

3.8.1 AI 在系统中的角色

本项目不能设计成：

代码块

```
1  User copy
2  → LLM
3  → "Is this compliant?"
```

核心架构应为：

代码块

```
1  Structured Knowledge
2  +
3  Deterministic Rules
4  +
5  Semantic Understanding
6  +
7  LLM Reasoning
```

3.8.2 两层 Decision Engine

Layer 1: Deterministic Obligation Check

负责：

代码块

```
1  必须有什么？
2  现在有没有？
```

典型内容：

- mandatory information;
- mandatory statements;
- required qualifier;
- applicable warning;
- required disclosure。

这一层优先使用结构化规则判断。

Layer 2: Semantic Claim Check

负责：

代码块

1 已经写出的内容意味着什么？

典型内容：

- absolute safety claim;
- therapeutic-style claim;
- evidence claim;
- time-bound effect claim;
- misleading absolute claim。

LLM 主要承担：

- claim extraction;
- semantic mapping;
- context interpretation;
- rewrite。

3.8.3 Decision Logic

基础逻辑：

代码块

```
1 IF required context missing
2   → Needs your input
3
4 ELSE IF regulatory boundary conflict
5   → Needs your input
6
7 ELSE IF decision depends on unavailable evidence
8   → Needs your input
9
10 ELSE IF explicit legal obligation is applicable
11   AND requirement is missing
12   → Fix before running
13
14 ELSE IF clear prohibited / unsupported claim detected
```

```
15 → Fix before running
16
17 ELSE
18 → Ready to run
```

3.8.4 AI 输入结构

代码块

```
1  content_id:
2  generated_copy:
3
4  country:
5  industry:
6
7  product:
8    regulation_type:
9    aust_l_number:
10   product_name:
11
12  channel:
13  content_format:
14
15  available_evidence:
16
17  knowledge_scope:
18  rule_version:
```

3.8.5 AI 输出结构

代码块

```
1  overall_status:
2
3  checks:
4    - flag_id:
5      text_span:
6      issue_type:
7      status:
8      rule_id:
9      reason:
```

```
10     source:
11     required_action:
12     suggested_rewrite:
13     delta:
14
15 questions:
16   - question_id:
17     reason:
18     required_field:
```

3.9 风险与边界

Risk 1：产品不能提供法律保证

不得输出：

代码块

```
1 This advertisement is legally compliant.
```

允许：

代码块

```
1 Ready to run
2 No required fix was identified within
3 the selected compliance scope.
```

原调研也明确指出不能承诺法律正确，高风险结论需要支持人工复核。filecite0turn0file0L82-L90

Risk 2：错误产品分类导致错误规则集

解决：

代码块

```
1 Auto-fill
2 +
3 Visible
4 +
```

```
5 Editable
6 +
7 Confirmable
```

Risk 3: False Positive

这是 MVP 最关键风险之一。

不能使用：

代码块

```
1 检测问题越多 = 产品越好
```

原调研已经将合规文案误杀率定义为重要 Guardrail。 [filecite](#) [turn0file0](#) [L49-L64](#)

Risk 4: False Negative

危险内容被错误放行。

系统失败或知识不足时，不得默认：

代码块

```
1 Ready to run
```

Risk 5: 法规变化

所有 Rule 必须保存：

代码块

```
1 rule_id:
2 version:
3 effective_date:
4 source:
5 last_reviewed_at:
```

Risk 6：用户反馈污染规则

用户点击：

代码块

```
1 flag_disputed
```

不能自动改变法规知识。

正确流程：

代码块

```
1 Dispute
2 → Review Queue
3 → Human Adjudication
4 → Confirm FP / Reject dispute
5 → Dataset update
```

Risk 7：Knowledge Base Scope Creep

项目容易错误地把重点转成：

“先把所有法规都建成知识库。”

这是本项目必须避免的。

优先级：

代码块

```
1 P0 Compliance Detection Pipeline
2 ↓
3 P1 Minimum Knowledge Infrastructure
4 ↓
5 P2 Knowledge Expansion
```

知识库服务检测，不是产品本体。

3.10 后续迭代方向

Phase 1

代码块

```
1  Australia
2  +
3  Beauty
4  +
5  Text
```

证明：

代码块

```
1  能发现明确问题
2  +
3  能发现缺失义务
4  +
5  不乱猜
6  +
7  控制误杀
```

Phase 2

增加：

- ARTG deeper integration;
- permitted indication mapping;
- evidence workflow;
- batch copy review;
- team review;
- audit history。

Phase 3

增加渠道政策：

代码块

- 1 Jurisdiction Check
- 2 +
- 3 Channel Check

UI 必须始终独立展示。

例如：

代码块

- 1 Australian law
- 2 Ready to run
- 3
- 4 Meta policy
- 5 Fix before running

Phase 4

扩展：

代码块

- 1 Image
- 2 Video
- 3 Landing page
- 4 Product page
- 5 Full campaign

Phase 5

知识库按统一 Schema 扩展：

代码块

- 1 Country
- 2 |— Australia
- 3 |— UK
- 4 |— US
- 5 |— ...
- 6
- 7 Industry
- 8 |— Beauty

```
9  |— Retail
10 |— Food
11 |— Finance
12 |— ...
13
14 Channel
15 |— Meta
16 |— TikTok
17 |— Google
18 |— ...
```

4. 系统架构设计

4.1 架构定位：多维度合规知识库系统

本项目后台本质上不是单一敏感词库，也不是为每个国家单独复制一套检测系统，而是一套可扩展的 **Multi-dimensional Compliance Knowledge System**。

系统按照多个维度组织合规知识，包括：

代码块

```
1  Country
2  +
3  Industry
4  +
5  Product Type
6  +
7  Claim Type
8  +
9  Rule Type
10 +
11 Channel
```

用于管理不同国家、行业和产品场景下对应的：

- 法律法规；
- 强制披露要求；
- 禁止性规则；
- 证据要求；
- 官方解释；

- 监管案例；
- 正向案例；
- Hard Negative；
- False Positive 样本；
- 历史规则版本。

核心原则是：

统一检测框架，不同场景加载不同规则集。

例如：

代码块

```
1  Australia
2  |—— Beauty
3  |—— Medical / Healthcare
4  |—— Fitness
5  |—— Baby / Parenting
```

未来扩展中国市场时，不需要重新开发一套产品，而是在统一架构下新增：

代码块

```
1  China
2  |—— Beauty
3  |—— Medical / Healthcare
4  |—— Fitness
5  |—— Baby / Parenting
```

因此项目扩展方式不是：

代码块

```
1  Australia Product
2  China Product
3  US Product
```

而是：

代码块

```
1  One Compliance Engine
2  +
```

4.2 为什么不能只做“国家词库”

国家只是规则路由的第一层。

即使处于同一个国家，不同行业、产品类型和营销声称所适用的规则也可能不同。

例如在 Australia 下：

代码块

```
1  Australia
2  |
3  |— Beauty
4  |   |— Cosmetic
5  |   |— AUST L Listed Medicine
6  |
7  |— Medical / Healthcare
8  |
9  |— Fitness
10 |
11 |— Baby / Parenting
```

不同场景可能关注完全不同的问题。

Beauty

可能关注：

- Cosmetic claims;
- Therapeutic claims;
- Mandatory information;
- Natural / organic claims;
- Evidence-supported efficacy claims;
- Sunscreen-related requirements.

Medical / Healthcare

可能关注：

- Treatment claims;

- Disease claims;
- Professional endorsement;
- Medical evidence;
- Mandatory warning;
- Restricted representation。

Fitness

可能关注：

- Performance improvement;
- Body transformation;
- Guaranteed results;
- Time-bound effects;
- Before / after claims;
- Evidence claims。

Baby / Parenting

可能关注：

- Infant safety;
- Development claims;
- Health claims;
- Age restrictions;
- Absolute safety language;
- Sensitive audience requirements。

因此不能简单使用：

代码块

```
1 keyword = "clinically proven"  
2 → Risk
```

而需要结合：

代码块

```
1 Country  
2 +
```

```
3   Industry
4   +
5   Product Type
6   +
7   Claim
8   +
9   Context
10  → Applicable Rule
```

同一句广告，在不同产品和行业中可能得到不同结果。

4.3 Base Rules + Industry Extension

为了降低知识维护成本，不同垂直行业不应完整复制一份法规知识库。

系统采用：

Base Rules + Industry Extension

的知识组织方式。

例如 Australia：

```
代码块
1   Australia Base Rules
2   |
3   |— Consumer protection
4   |— Misleading claims
5   |— Evidence principles
6   |— General advertising obligations
7       |
8       |— Beauty Extension
9       |— Medical Extension
10      |— Fitness Extension
11      |— Baby Extension
```

Base Rules

保存多个行业共同适用的基础规则。

例如：

```
代码块
1   Australia
```


该规则可以同时被 Beauty、Fitness、Retail 等行业调用。

Industry Extension

保存特定行业才适用的特殊规则。

例如：

代码块

- 1 Australia
- 2 → Beauty
- 3 → AUST L Listed Medicine
- 4 → Mandatory statements

只有符合相应产品 Context 时才加载。

Product Type Extension

部分行业内部还需要继续按照产品监管身份拆分。

例如：

代码块

- 1 Australia
 - 2 └ Beauty
 - 3 └ Cosmetic
 - 4 └ ACL-based claim check
 - 5 └
 - 6 └ AUST L Listed Medicine
 - 7 └ Mandatory obligations
 - 8 └ Permitted indications
 - 9 └ Therapeutic advertising rules

这样可以避免：

代码块

- 1 Beauty

导致大量误杀。

4.4 多维 Knowledge Routing

知识库本身不直接完成检测。

用户内容进入系统后，首先通过 **Knowledge Router** 判断应该调用哪一个知识范围。

整体过程：

代码块

```
1  Generated Content
2      ↓
3  Context Collection
4      ↓
5  Knowledge Routing
6      ↓
7  Applicable Rule Set
8      ↓
9  Decision Engine
```

Routing Input

至少包括：

代码块

```
1  country:
2  industry:
3  product_type:
4  product_regulation:
5  claim_type:
6  channel:
7  content_format:
```

例如：

代码块

```
1  country: Australia
2  industry: Beauty
3  product_regulation: AUST_L
```

```
4  claim_type: sunscreen_claim
5  channel: Instagram
```

系统才会进一步调用：

代码块

```
1  Australia
2  → Beauty
3  → AUST L
4  → Sunscreen Rules
```

而不是检索整个法规知识库。

Routing 原则

原则一：优先使用用户已确认信息

如果用户已经明确选择：

代码块

```
1  Product = Cosmetic
```

系统不能静默改成：

代码块

```
1  AUST L
```

原则二：自动识别必须可见、可修改

正式产品可以利用 Canva 已掌握的信息自动预填：

代码块

```
1  Country
2  Industry
3  Product Type
4  Channel
```

但 UI 必须明确展示当前 Context。

例如：

代码块

1

Market

2

Australia ▼

3

4

Product type

5

Cosmetic ▼

用户可以一键修改。

原则是：

|

Auto-fill is allowed. Hidden assumptions are not.

原则三：无法确定时追问

如果系统无法可靠确定：

代码块

1

Cosmetic

2

or

3

Therapeutic Good

不得自行猜测。

进入：

代码块

1

Needs your input

并向用户提问。

原则四：发现分类冲突时停止继续推断

例如：

代码块

1

User confirmed:

但文案中出现明显 therapeutic-style claim。

系统触发：

代码块

```
1 Regulatory Boundary Trigger
```

输出：

代码块

```
1 Needs your input
```

提示用户重新确认产品监管身份。

而不是自动切换 TGA 规则。

4.5 Knowledge Schema

所有国家和行业使用统一 Schema。

例如：

代码块

```
1 rule_id:
2
3 country:
4 industry:
5 product_type:
6 product_regulation:
7
8 claim_type:
9
10 rule_type:
11   - mandatory
12   - prohibited
13   - evidence_required
14   - conditional
15   - exception
16
```

```
17  applicability_condition:
18
19  trigger_condition:
20
21  exception_condition:
22
23  required_content:
24
25  decision_logic:
26
27  action_status:
28    - ready_to_run
29    - fix_before_running
30    - needs_your_input
31
32  official_source:
33
34  source_url:
35
36  effective_date:
37
38  rule_version:
39
40  examples:
41    positive:
42    negative:
43    hard_negative:
44
45  false_positive_samples:
46
47  last_reviewed_at:
48
49  review_status:
```

Schema 设计目的

统一 Schema 可以保证未来增加：

代码块

- 1 Australia → Beauty
- 2 China → Beauty
- 3 Australia → Fitness
- 4 UK → Baby

时，不需要改变核心检测 Pipeline。

新增市场本质上变成：

代码块

```
1  新增 Knowledge Package
```

而不是：

代码块

```
1  重新开发一个产品
```

4.6 Rule Version Management

法规会发生变化，因此知识库不能只保存“当前规则文本”。

每一条规则必须保存：

代码块

```
1  rule_id:
2  version:
3  effective_date:
4  expired_date:
5  official_source:
6  last_reviewed_at:
7  reviewer:
```

用户每次检测同时保存：

代码块

```
1  Which rule
2  +
3  Which version
```

例如：

代码块

```
1  rule_id: AU_TGA_0012
```

这样当企业之后查看历史记录时，可以回答：

当时系统是依据哪个版本的规则做出的判断？

这一能力同时支撑：

- Audit Trail；
- 企业审查记录；
- 法规更新回溯；
- 模型错误分析；
- 评测集回归测试。

4.7 Compliance Knowledge System 内容结构

后台知识系统建议管理以下六类核心内容。

1. Regulations

官方法规及监管要求。

例如：

代码块

- 1 Law
- 2 Code
- 3 Official Guidance
- 4 Mandatory Requirement

2. Structured Rules

将法规转换为机器可执行条件。

例如：

代码块

- 1 IF
- 2 Product = X
- 3
- 4 AND


```
5 Claim Type = Y
6
7 THEN
8 Required Information = Z
```

这是 Decision Engine 实际使用的主要内容。

3. Regulatory Cases

保存真实监管案例。

用于：

- 辅助解释；
 - 构造 Evaluation Case；
 - 验证规则适用边界。
-

4. Good Examples

明确安全或正常表达样本。

例如：

代码块

```
1 This is our team's favourite formula.
```

这类数据可以帮助模型学习：

| 什么情况下不应该报警。

5. Hard Negatives

Hard Negative 是本项目非常重要的一类知识数据。

定义为：

| 表面看起来像高风险广告，但在当前 Context 下实际上不应该被拦截的内容。

例如：

代码块

```
1 含有高风险关键词
```

2 但法规条件并不成立

Hard Negative 主要用于：

- 降低 False Positive；
- AI Evaluation；
- Regression Test。

6. False Positive Samples

来自真实使用过程中：

代码块

```
1 flag_disputed
2 ↓
3 Human Review
4 ↓
5 Confirmed False Positive
```

最终进入 FP Library。

流程必须是：

代码块

```
1 User Dispute
2 → Human Adjudication
3 → Confirmed False Positive
4 → Add to Dataset
```

而不能：

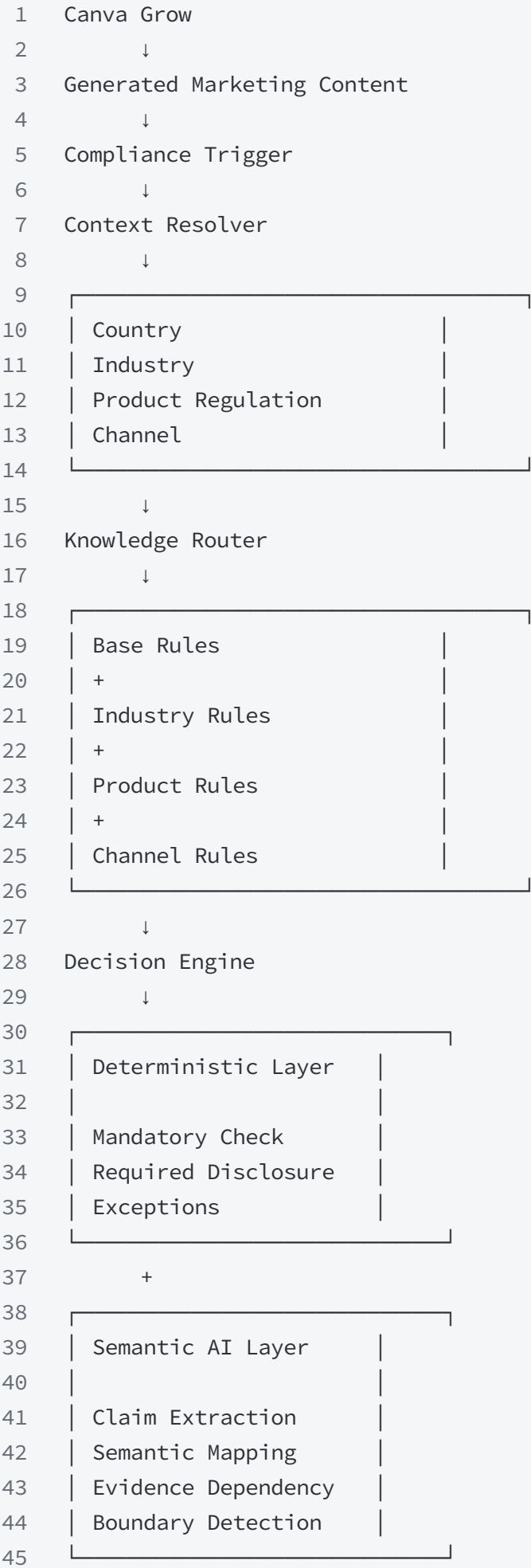
代码块

```
1 User Dispute
2 → Automatically change rule
```

4.8 检测架构

完整检测流程：

代码块



```

46      ↓
47    Decision Aggregator
48      ↓
49    ┌───────────────────────────────────────────────────────────────────────────────────┐
50    │ Ready to run                                                                    │
51    │ Fix before running                                                            │
52    │ Needs your input                                                             │
53    └───────────────────────────────────────────────────────────────────────────────────┘
54      ↓
55    Explanation
56    +
57    Official Source
58    +
59    Minimal Rewrite
60    +
61    Delta
62      ↓
63    User Decision
64      ↓
65    Re-check / Override / Dispute
66      ↓
67    Audit Log
68    +
69    Evaluation Dataset

```

4.9 法域与渠道独立

系统架构中必须将：

代码块

1 Jurisdiction Compliance

和：

代码块

1 Channel Policy Compliance

拆成两个独立维度。

禁止计算：

代码块
Overall Compliance Score = 78

因为一条广告可能：

代码块

- 1 Australian law
- 2 Ready to run
- 3
- 4 Meta policy
- 5 Fix before running

也可能：

代码块

- 1 Australian law
- 2 Fix before running
- 3
- 4 Meta policy
- 5 Ready to run

二者代表的是完全不同的问题。

因此数据结构建议：

代码块

- 1 jurisdiction_check:
- 2 country:
- 3 status:
- 4 rules:
- 5
- 6 channel_check:
- 7 channel:
- 8 status:
- 9 rules:

MVP 暂时只实现：

代码块

- 1 Jurisdiction Check

但系统架构必须提前保留：

代码块

1 Channel Check

接口位。

4.10 系统扩展方式

未来扩展新市场时：

扩展新国家

例如：

代码块

1 Australia
2 ↓
3 China

主要新增：

代码块

1 China Base Rules
2 +
3 China Industry Extensions

扩展新行业

例如从：

代码块

1 Beauty

增加：

1 Fitness

主要新增：

代码块

- 1 Fitness Rule Package
- 2 +
- 3 Fitness Hard Negatives
- 4 +
- 5 Fitness Cases

扩展新渠道

例如增加：

代码块

- 1 Meta
- 2 TikTok
- 3 Google

新增：

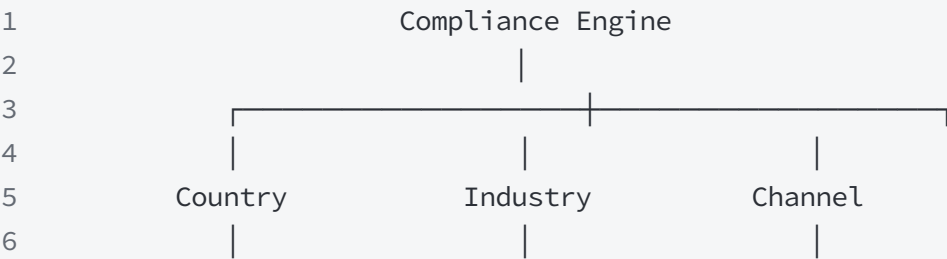
代码块

- 1 Channel Policy Package

不改变 Jurisdiction Engine。

最终扩展模型

代码块



7	Australia	Beauty	Meta
8	China	Fitness	TikTok
9	UK	Medical	Google
10	US	Baby	...

所有规则最终通过统一的：

代码块

```
1 Context
2 → Routing
3 → Rule Set
4 → Decision
5 → Action
```

流程执行。

4.11 核心研发优先级

虽然系统后台采用多维知识架构，但本项目一期核心不是建设“大而全的知识库”。

优先级必须保持：

代码块

```
1 P0
2 Compliance Detection Pipeline
```

包括：

- Context 获取；
- Knowledge Routing；
- Mandatory Check；
- Semantic Claim Check；
- Decision Logic；
- Needs your input；
- Rewrite；
- Re-check；
- Audit。

其次：

代码块

- 1 P1
- 2 Minimum Viable Knowledge System

只建立支撑 Australia + Beauty MVP 所需的最小规则集。

最后：

代码块

- 1 P2
- 2 Knowledge Expansion

逐步扩展：

代码块

- 1 More rules
- 2 → More industries
- 3 → More countries
- 4 → More channels

因此架构设计原则是：

先证明检测闭环有效，再扩大知识覆盖。

知识库是检测能力的基础设施，而不是一期产品目标本身。

5. 数据与埋点

5.1 核心事件

代码块

- 1 compliance_check_triggered
- 2 context_auto_filled
- 3 context_changed
- 4 context_confirmed
- 5
- 6 flag_generated
- 7 needs_input_triggered
- 8 user_input_provided

```
9
10  fix_accepted
11  fix_edited
12  override_confirmed
13  flag_disputed
14
15  recheck_completed
16  ready_to_run_reached
17  content_continued
```

5.2 为什么必须有 `flag_disputed`

不能使用：

代码块

```
1  用户没有接受 rewrite
```

作为误杀。

因为：

代码块

```
1  Keep Original
```

可能只是商业决策。

只有：

代码块

```
1  flag_disputed
```

表示用户明确挑战系统判断。

但：

| User Dispute $\neq$ Confirmed False Positive

最终仍需要人工复核。

5.3 北极星指标

建议沿用原调研逻辑：

完成合规检查，并最终被采用 / 继续使用的营销内容数。

原调研也明确不建议把警告数量或纯生成量作为成功指标。 [filecite](#) [turn0file0](#) [L63-L64](#)

6. AI Evaluation

参考 AI 工具助手 PRD，本项目同样必须在线上前通过统一评测集验证，而不能只根据 Demo 体验判断 AI 能力。参考 PRD 将正向、负向、参数缺失、未支持能力等都纳入测试。

[filecite](#) [turn1file0](#) [L251-L290](#)

6.1 MVP Evaluation Set

计划：

代码块

1 60 Cases

其中：

代码块

1 24 Hard Negative – 40%
2 18 True Fix
3 18 Needs Your Input

6.2 Hard Negative

目的：

验证系统知道什么时候不应该拦。

例如：

- 正常使用 `formula` ；
- 主观品牌偏好；
- 条件不满足某条法规时不应触发；
- exception 生效；

- 类似高风险 Claim，但实际 Context 不适用。
-

6.3 True Fix

验证：

信息已经充分时，明确问题是否能够稳定识别。

包括：

- required information missing;
 - clear prohibited claim;
 - mandatory obligation not met.
-

6.4 Needs Your Input

验证：

AI 是否能在“不知道”的情况下正确停下来。

包括：

- 产品监管类型未知;
 - claim 与产品身份冲突;
 - evidence unavailable;
 - AUST L / product information missing;
 - 判断依赖外部事实。
-

6.5 Acceptance Criteria

以下指标必须设上线门槛：

指标	定义	MVP Threshold
Fix Recall	应进入 Fix 的 Case 被正确识别	TBD
Hard Negative FP Rate	合法 Hard Negative 被误判为 Fix 的比例	TBD
Needs Input Routing Accuracy	应追问的 Case 是否正确进入 Needs your input	TBD
Citation Correctness	引用规则是否与当前判断匹配	TBD
Rewrite Validity	Rewrite 是否解决原问题	TBD
Delta Accuracy	系统是否准确说明修改内容	TBD

这里暂不虚构具体百分比。
下一步需要结合第一版 60 条 Benchmark 跑出的 Baseline 再设最终上线红线。

7. 当前已验证与待验证内容

已有证据

当前 Grow 截图可以支持：

- 1. Grow 已存在“用户输入 → 生成多个广告概念 → 继续广告生成”的真实流程；
- 2. 美妆内容中存在时间效果型表达，例如“14 天”等 Claim；
- 3. 测试中已经主动输入过“No side effects / 纯天然”等高风险表达；
- 4. “团队最爱的一款配方”等文本可作为 Hard Negative 候选；
- 5. 当前 AI 生成工作流本身并没有展示独立的法域 Compliance 判断过程。

尚未作为一手证据确认

当前材料还不能直接证明：

代码块

1 Grow 实际生成了一条

- 2 AUST L sunscreen advertisement
- 3 并遗漏 specific mandatory information

因此 PRD 中该类案例目前应该定义为：

代码块

- 1 Planned Validation Case

而不是：

代码块

- 1 Observed Production Failure

待找到对应 SolGuard SPF 50+ 实测材料后，再升级为真实 Observed Bad Case。

8. 产品核心原则总结

整个项目最终不是：

一个更聪明的敏感词工具。

而是建立：

代码块

- 1 Context
- 2 ↓
- 3 Applicable Rule
- 4 ↓
- 5 What was said?
- 6 ↓
- 7 What was not said?
- 8 ↓
- 9 What facts are missing?
- 10 ↓
- 11 What should the user do next?

四条核心原则：

8.1.1 Compliance appears before the user remembers to ask.

合规检查必须进入内容生产链路。

8.1.2 Ask, don't guess.

判断依赖用户事实时，正式进入 `Needs your input`。

8.1.3 Missing information is also a compliance problem.

不仅检测“说错了什么”，还检测“应该说但没有说什么”。

8.1.4 False positives are a product failure.

不是拦得越多越好。

产品最终需要证明的不是：

“AI 可以发现风险。”

而是：

它知道什么时候应该拦、什么时候应该问、什么时候应该放行。